Replication

什麼是 Replication？

Replication（複製）的概念很直觀：把同一份資料的副本，放在多台透過網路連接的機器上。但為什麼要這樣做？原因有三個：

- **降低延遲**：把資料放在地理上靠近用戶的地方，讀取更快
- **提升可用性**：某些節點失效時，系統仍能繼續運作
- **擴展讀取的吞吐量**：把讀取請求分散到多台機器上

聽起來很簡單，但難就難在資料會改變。如果資料永遠不變，replication 只需要把資料複製到每個節點一次就完工了。真正的挑戰在於：當資料持續更新時，如何讓所有副本保持一致。

這篇講義會介紹三種主要的 replication 架構：**Single-leader**（單一主節點）、**Multi-leader**（多主節點）、**Leaderless**（無主節點）。每種都有各自的優缺點，也都有適合的應用場景。

第一種架構：Single-Leader Replication

這是最常見、也最直覺的架構。

基本運作方式

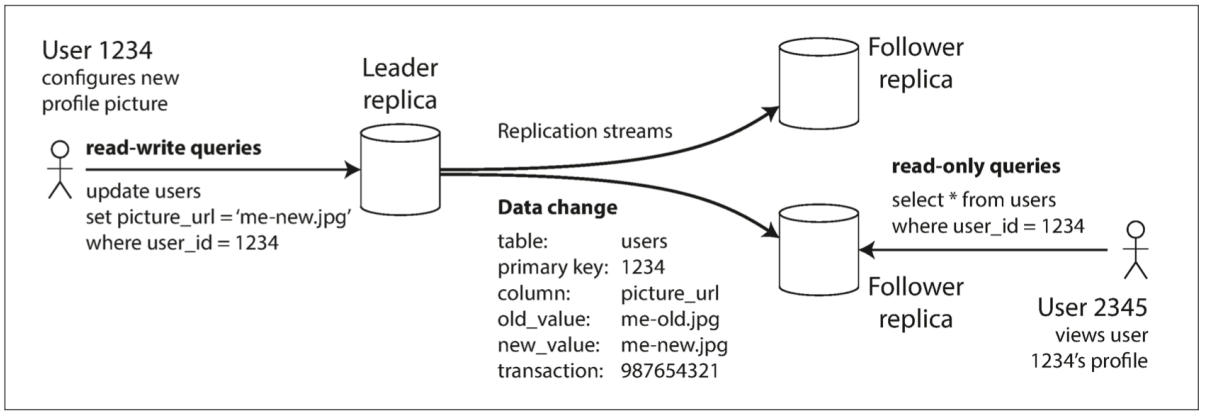
每個儲存資料副本的節點叫做 **replica（副本）**。有了多個 replica，問題就來了：怎麼確保所有 replica 的資料都一樣？

Leader-based replication 的運作如下：

1. 其中一台 replica 被指定為 **leader（主節點）**，也叫 master 或 primary。所有的寫入都必須先打到 leader。
2. 其他 replica 稱為 **follower（從節點）**，也叫 slave、secondary 或 read replica。Leader 把每次寫入包成 replication log，發送給所有 follower，follower 照順序套用這些變更。
3. 讀取可以打到 leader 或任何 follower，但寫入只能打到 leader。

buildmoat.org

這個模式被 PostgreSQL（9.0 以後）、MySQL、MongoDB、Kafka、RabbitMQ 等廣泛採用。



同步 vs. 非同步 Replication

這是整個 replication 最關鍵的取捨之一。

同步 replication（Synchronous）：Leader 在確認 follower 已收到並回報成功後，才通知 client 寫入完成。好處是 follower 保證有最新資料；壞處是一旦那台 follower 沒回應，整個系統的寫入就全部卡住了。

非同步 replication（Asynchronous）：Leader 把訊息發出去，不等 follower 確認就通知 client 成功。寫入更快，但如果 leader 在 follower 同步之前就掛了，那些寫入就永久消失了。

在實際的系統裡，讓所有 follower 都同步是不現實的，任何一個節點出問題就會讓整個系統停擺。常見的做法是**半同步（Semi-synchronous）**：保持一台 follower 同步、其他都非同步。這樣至少有兩份節點（leader + 一台 follower）保有最新資料。

> **面試提示**：同步與非同步 replication 的取捨是面試必考點。同步確保一致性但犧牲可用性；非同步確保可用性但有資料遺失風險。沒有哪個更好，而是要看你的系統對一致性和可用性的要求。

新增 Follower

有時候你需要加入新的 follower，可能是為了增加讀取容量，或替換掛掉的節點。你不能只是複製檔案，因為資料庫一直在寫入，複製過程中的資料會不一致。

不用停機的標準流程如下：

1. 在某個時間點對 leader 的資料庫拍一個一致的快照（snapshot），通常不需要鎖定整個資料庫
2. 把快照複製到新的 follower 節點

buildnoat.org

3. Follower 連接到 leader，請求自快照以來發生的所有資料變更（每個快照都對應 replication log 中的一個精確位置，在 PostgreSQL 叫 log sequence number，在 MySQL 叫 binlog coordinates）
4. Follower 追上這段還沒被處理的變更之後，就可以繼續即時處理 leader 的資料變更

處理節點失效

Follower 失效：追趕恢復

每台 follower 在本地磁碟上保留一份 replication log。如果 follower 崩潰後重啟，它知道上次處理到哪裡，直接向 leader 請求中斷期間的所有變更，追上之後繼續即時接收。

Leader 失效：Failover

Leader 掛掉比較棘手：必須把某台 follower 提升為新的 leader，讓 client 把寫入打去新 leader，其他 follower 也開始從新 leader 接收資料。這個過程叫 **failover**。

自動 failover 的步驟通常是：

1. **偵測 leader 失效**：沒有萬無一失的方法，大多數系統用 timeout，如果節點超過一段時間（例如 30 秒）沒回應，就認定它掛了
2. **選出新 leader**：透過選舉（由多數 replica 投票），或由之前選出的 controller 節點指定。最好選資料最新的那台 replica
3. **重新設定系統**：讓 client 的寫入打去新 leader；如果舊 leader 回來，確保它成為 follower，認識新的 leader

Failover 的地雷：

- 如果用非同步 replication，新 leader 可能還沒收到舊 leader 最後的一些寫入，這些寫入通常會被丟棄，違反 client 對持久性的期待
- **Split brain**：如果兩個節點都以為自己是 leader，兩邊都接受寫入，資料可能損毀。有些系統用 STONITH（Shoot The Other Node In The Head）機制關掉其中一個，但如果設計不好，可能兩個都被關掉
- Timeout 的長短很難拿捏：太長，失效後恢復慢；太短，可能在系統本來就很忙的時候不必要地觸發 failover，讓情況更糟

Replication Log 的實作方式

Statement-based replication（語句複製）：Leader 把每個 SQL 語句（INSERT、UPDATE、DELETE）發給 follower 執行。問題在於非確定性函數（如 `NOW()`、

buildneat.org

`RAND()`）在不同 replica 上會產生不同結果，還有 autoincrement 和有副作用的語句都可能導致不一致。MySQL 5.1 之前用這個方式。

Write-ahead log (WAL) shipping（預寫日誌傳送）：把 storage engine 用來做崩潰恢復的那份 WAL 直接傳給 follower。缺點是 WAL 在非常底層的位元組層次描述資料，replication 和 storage engine 緊密耦合，資料庫升版本時可能需要停機。PostgreSQL 和 Oracle 使用這個方式。

Logical (row-based) log replication（邏輯日誌複製）：用一種和 storage engine 分離的格式記錄 row 層次的變更（插入記錄新值、刪除記錄識別 row 的資訊、更新記錄識別資訊和新值）。這讓 leader 和 follower 可以跑不同版本，格式也更容易被外部系統解析，這就是 **Change Data Capture（CDC）** 的基礎。MySQL 的 binlog 使用這個方式。

Trigger-based replication（觸發器複製）：把 replication 邏輯搬到應用層，用 trigger 和 stored procedure 捕捉變更，允許更靈活的客製化。Overhead 更高，但在需要只複製部分資料、或不同類型資料庫間互相複製時有用。

Replication Lag 的問題

Single-leader replication 在讀取遠多於寫入的場景很有吸引力，像是加更多 follower，把讀取分散出去。但這只有在非同步 replication 下才現實（全同步的話，任何一台 follower 失效都會讓整個系統無法寫入）。

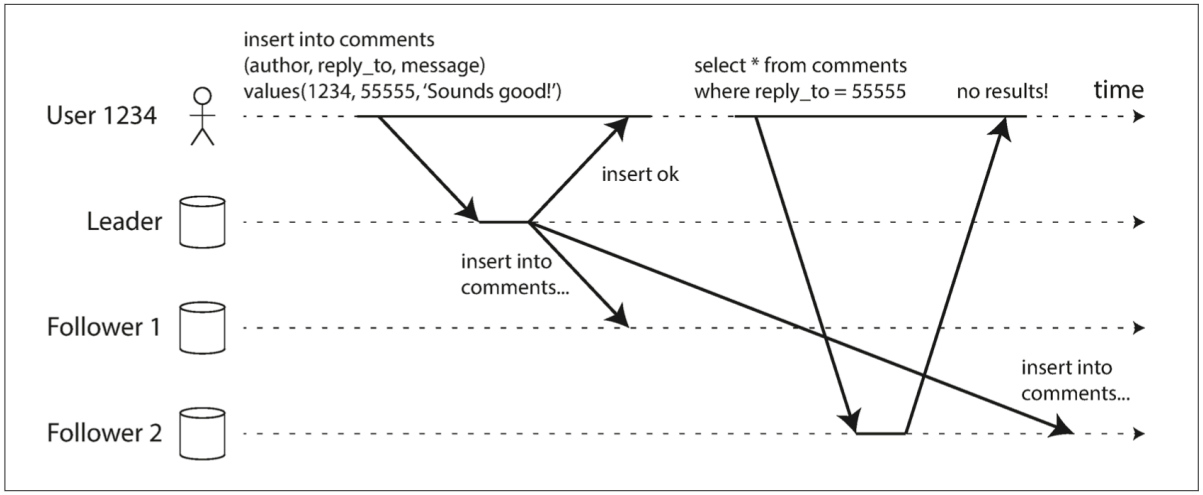
非同步 replication 的代價是：從 follower 讀取可能會看到過時的資料。Leader 剛寫入、Follower 還沒同步時，對同一個 query，leader 和 follower 的回答可能不一樣。這種暫時的不一致叫做 **eventual consistency（最終一致性）**，如果停止寫入，follower 最終會追上 leader。

「最終」這個詞刻意模糊，lag 通常只有幾毫秒，但在系統接近滿載或網路有問題時，可能拉長到幾秒甚至幾分鐘。

以下是 replication lag 造成的三種常見問題，以及解法。

問題一：Read-After-Write（讀自己剛寫的）不一致

buildpost.org



用戶發布了一則貼文，馬上去看，卻看不到，因為他的讀取打到了一台還沒同步的 follower。

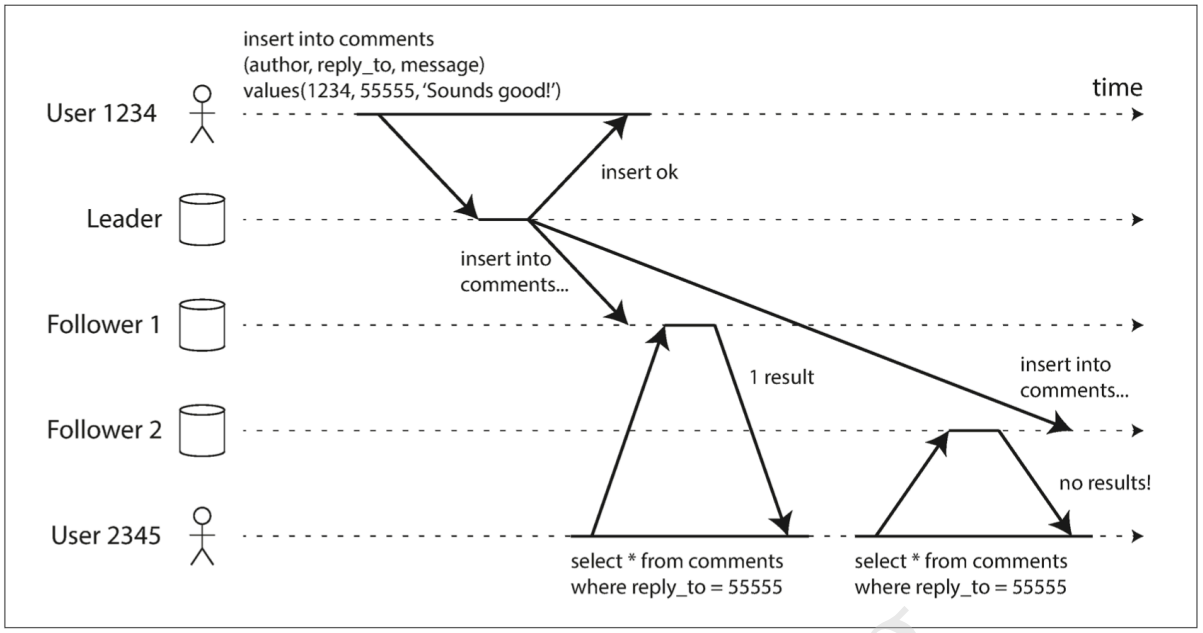
解法：

- **讀自己可能改過的東西時，從 leader 讀**。比如社群媒體上，用戶自己的 profile 只有自己能改，所以自己看自己 profile 時永遠從 leader 讀，看別人的 profile 從 follower 讀。
- **追蹤最後一次更新的時間**，如果距離不夠遠，就從 leader 讀，或拒絕從太落後的 follower 讀。
- **Client 記住最後一次寫入的 timestamp**，系統確保讀取時 replica 的資料至少和這個 timestamp 一樣新。

多裝置的情境更複雜，你在手機改了什麼，從電腦去讀，兩台裝置的 timestamp 需要中央同步，而且可能被路由到不同的 datacenter。

問題二：Monotonic Reads（單調讀取）不一致

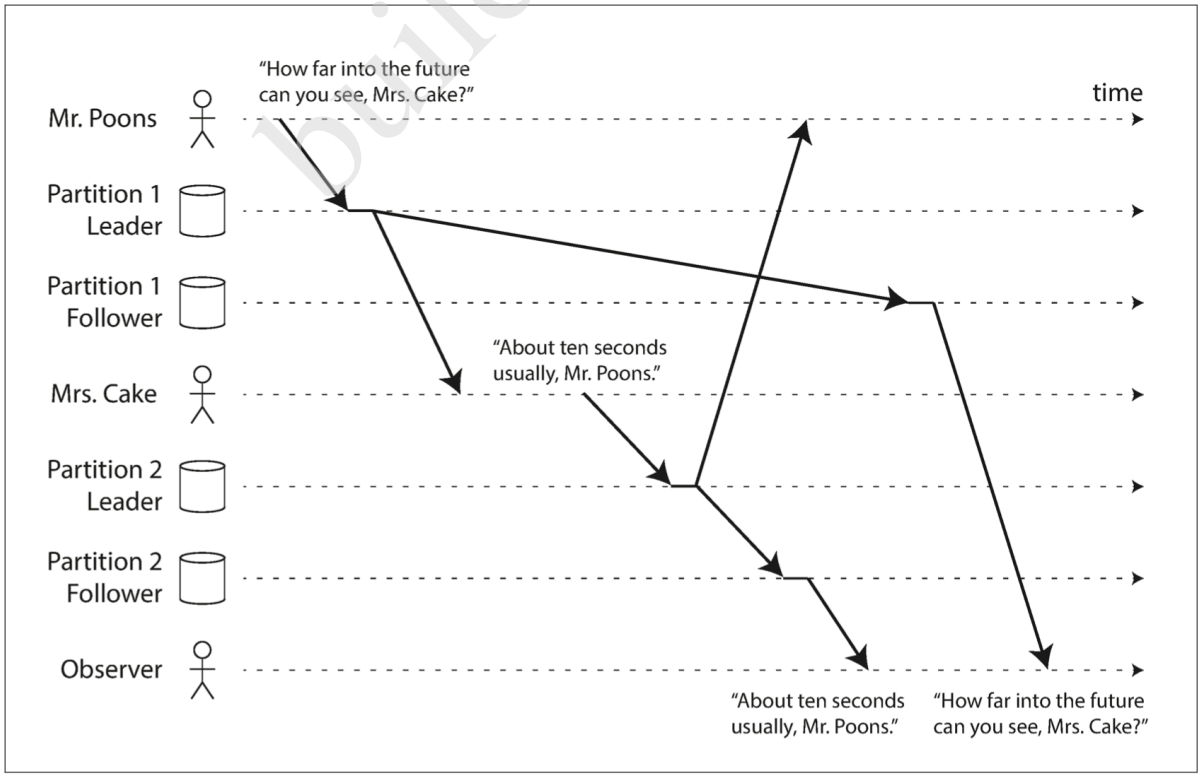
buildmoat.org



用戶先從一台 lag 小的 follower 讀到了朋友的留言，然後刷新頁面，被路由到一台 lag 大的 follower，留言消失了，看起來像時光倒流。

解法：確保同一個用戶的讀取永遠打到同一台 replica。可以用用戶 ID 的 hash 來決定要讀哪台，不同用戶可以讀不同 replica，但同一個用戶總是讀同一台。如果那台 replica 掛了，再路由到另一台。

問題三：Consistent Prefix Reads（因果一致性）不一致



buildmoat.org

A 問了一個問題，B 回答了。第三個觀察者因為 replication lag 不同，先看到 B 的回答，才看到 A 的問題，因果順序顛倒了。

解法：確保有因果關係的寫入都寫到同一個 partition。但不是所有應用都能高效地做到這點；另一種做法是使用記錄因果依賴關係的演算法（version vector）。

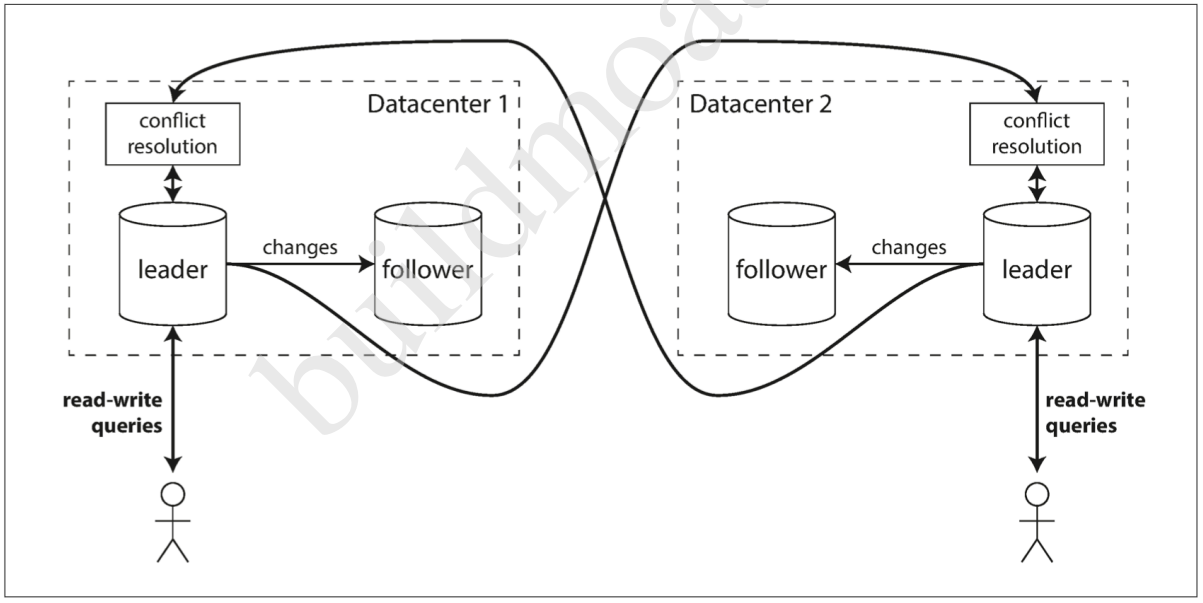
核心洞察

與其讓應用程式自己處理這些微妙的 replication 問題，更好的做法是用 transaction 讓資料庫提供更強的保證。但分散式系統中的 transaction 有效能和可用性的代價，沒有一個放之四海皆準的答案。

第二種架構：Multi-Leader Replication

Single-leader 有一個根本的弱點：所有寫入都得通過那一台 leader。連不到 leader，就寫不進去。

Multi-leader replication 讓多個節點都能接受寫入。每個 leader 同時也是其他 leader 的 follower，把自己的寫入複製給其他所有 leader 和它自己的 follower。



> **面試提示**：在單一 datacenter 內使用 multi-leader 幾乎從不值得，複雜度遠超過好處。但跨多個 datacenter 的場景就完全不同了。

Multi-Leader 的適用場景

多 datacenter 部署：每個 datacenter 有自己的 leader，datacenter 內部用一般的 leader-follower replication，datacenter 之間各 leader 互相非同步複製。

與 single-leader 比較：

buildmoat.org

面向	Single-Leader	Multi-Leader
效能	每次寫入都要跨越網路到 leader 的 datacenter，增加延遲	寫入在本地 datacenter 處理，跨 datacenter 的延遲對用戶不可見
Datacenter 失效容忍	Leader 所在的 datacenter 掛了就要 failover	每個 datacenter 可以獨立運作，等到失效的 datacenter 恢復時再同步
網路問題容忍	對 datacenter 間連線的問題很敏感	非同步 replication 可以容忍暫時的網路中斷

離線操作：手機日曆 App 在沒有網路時也能繼續新增行程，等連上網再同步。每台裝置都是一個 leader，同步就是 multi-leader replication，lag 可能是幾小時甚至幾天。

協作編輯：Google Docs 讓多人同時編輯同一份文件。每個用戶的本地副本是一個 leader，變更即時套用到本地，非同步複製給其他人。這帶來了和 multi-leader replication 完全一樣的寫入衝突問題。

處理寫入衝突

這是 multi-leader replication 最大的挑戰。如果用戶 A 和用戶 B 同時在不同的 leader 修改同一份資料，都成功了，但這兩個寫入在非同步複製時就會衝突。

衝突迴避（Conflict Avoidance）：最簡單的方法是確保特定記錄的所有寫入都走同一個 leader，衝突就不會發生。比如把同一個用戶的請求永遠路由到同一個 datacenter。但如果那個 datacenter 掛了，就必須改路由，衝突迴避就失效了。

收斂到一致狀態（Converging）：資料庫必須讓所有 replica 最終達到相同的值。幾種方法：

- **Last Write Wins (LWW)**：每個寫入附上 timestamp，挑 timestamp 最大的當勝者，丟棄其他寫入。實作簡單，但會丟失資料。這是 Cassandra 唯一支援的衝突解法，Riak 也有這個選項。
- **Replica 優先序**：編號較高的 replica 的寫入永遠優先。同樣會丟失資料。
- **合併值**：把衝突的值合在一起（例如按字母排序後串接）。
- **記錄衝突、稍後解決**：把所有衝突版本都保留，等應用程式或用戶決定怎麼解決。

自訂衝突解法（Custom Conflict Resolution Logic）：

- **On write**：偵測到衝突時立刻呼叫衝突處理函數（background process 執行）
- **On read**：保留所有衝突的版本，下次讀取時全部回傳給應用程式，讓應用程式決定

自動衝突解法的研究方向：

buildmeat.org

- **CRDT（Conflict-free Replicated Datatypes）**：可以被多人同時編輯、並自動以合理方式解決衝突的資料結構（集合、計數器、有序列表等）
- **Mergeable Persistent Data Structures**：類似 Git，顯式追蹤歷史，用三方合併（three-way merge）
- **Operational Transformation**：Etherpad 和 Google Docs 使用的演算法，專為並發編輯有序字元列表設計

Multi-Leader 的 Replication Topology

Topology 描述寫入在 leader 之間如何傳播：

- **Circular topology**：每個節點只從一個節點接收，再轉發給下一個（MySQL 的預設）
- **Star topology**：一個根節點把寫入轉發給所有其他節點
- **All-to-all topology**：每個 leader 把寫入發給所有其他 leader

Circular 和 star topology 的問題：任何一個節點掛掉，就會中斷 replication 的流動。All-to-all 的容錯性更好，但可能出現寫入到達不同 replica 的順序不一致的問題（causality 違反）——這需要 version vector 來解決。

第三種架構：Leaderless Replication

完全拋棄 leader 的概念，讓任何 replica 都能直接接受寫入。Amazon 的內部 Dynamo 系統讓這個架構重新流行起來，Riak、Cassandra、Voldemort 都受到這個設計啟發。

當節點掛掉時怎麼寫入

想像一個有三個 replica 的系統，其中一台暫時不可用。

在 leaderless 系統裡，client 把寫入同時發給所有三台 replica。如果兩台成功回應，就算寫入成功（兩台掌聲就夠了），client 直接忽略那台沒回應的 replica。

問題來了：那台 replica 恢復後，它的資料已經過時。如果 client 從它讀取，就會拿到舊的值。

解法：讀取時，client 也同時向多台 replica 發送讀取請求，用 version number 判斷哪個回應更新，以最新的為準。

讓 Stale Replica 追上進度

Read repair（讀取修復）：Client 並行讀取多台 replica，發現某台的 version 比較舊，就把新值寫回那台。這對被頻繁讀取的值效果很好。

buildnote.org

Anti-entropy process（反熵程序）：背景程序持續掃描 replica 間的差異，把缺少的資料複製過去。不像 leader-based replication 的 replication log 有順序，這個程序不保證複製順序，也可能有明顯的延遲。

Quorum Reads 和 Writes

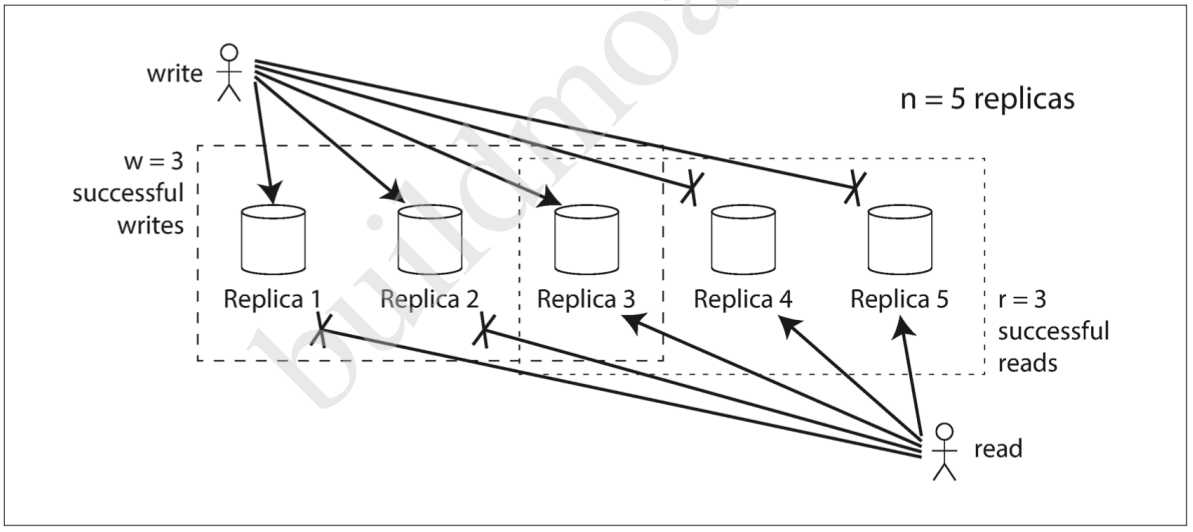
假設有 **n** 個 replica，寫入需要 **w** 個節點確認，讀取需要查詢 **r** 個節點。只要 **w + r > n**，讀取時至少有一個節點有最新資料，因為 w 個寫入節點和 r 個讀取節點必定有重疊。

這些滿足條件的讀寫操作叫做 **quorum reads 和 writes**。

常見配置：n = 3, w = 2, r = 2（容忍一個節點失效）；或 n = 5, w = 3, r = 3（容忍兩個節點失效）。

調整取捨：

- 少讀取多寫入的系統：可以設 w = n, r = 1，讀取超快，但只要一個節點掛掉寫入就失敗
- 多讀取少寫入：相反的配置



Quorum 一致性的限制

即使滿足 w + r > n，在某些邊緣情況下仍可能讀到過時值：

- **Sloppy quorum**：如果用了 sloppy quorum（見下節），w 個寫入節點和 r 個讀取節點可能完全沒有重疊
- **並發寫入**：兩個寫入同時發生，不清楚哪個先如果用 LWW，因為 clock skew 可能丟失寫入
- **並發讀寫**：寫入只反映在部分 replica，讀取結果不確定

buildmoat.org

- **節點失效 + 舊資料恢復**：持有新值的節點掛了，從舊副本恢復，保有新值的 replica 數量可能低於 w

Dynamo-style 的資料庫主要針對能接受最終一致性的場景優化。Quorum 不提供讀自己的寫入、單調讀取、consistent prefix reads 這些保證，如果需要更強的一致性，要用 transaction 或 consensus。

Sloppy Quorum 和 Hinted Handoff

想像一個大型 cluster，網路中斷讓 client 連不到應該存這份資料的 n 台「家」節點，但還連得到其他節點。

兩個選擇：

1. 返回錯誤
2. **Sloppy quorum**：接受寫入，暫時存到現在連得到的節點上（不在原本的 n 台裡）

Sloppy quorum 提升了寫入可用性，只要任何 w 台節點可達，就能寫入。但代價是即使 w + r > n，也無法保證讀到最新值，因為最新值可能在那些「臨時保管」的節點上。

網路恢復後，這些臨時保管的寫入會被送回「家」節點，這個過程叫 **hinted handoff（帶提示的交接）**。就像你因為沒帶鑰匙借住鄰居家，等拿到鑰匙後再回自己家。

偵測並發寫入

Leaderless 系統（以及 multi-leader 系統）允許多個 client 並發寫入同一個 key，衝突無可避免。

Last Write Wins (LWW)：強制給每個寫入附上 timestamp，取最大的當最終值。達成最終一致，但犧牲持久性，並發的多個寫入只有一個會存活，其他被靜默丟棄。只在可以接受資料遺失的場景（如 caching）才適用。

Happens-before 關係：如果操作 B 知道操作 A 的存在、依賴 A、或建立在 A 之上，就說 A 先於 B。如果兩個操作都不知道對方，就是**並發**（concurrent）。並發不是指「完全在同一個時間點」，而是「互不知情」。

用 Version Vector 追蹤因果依賴：

每個 replica 為每個 key 維護自己的 version number。操作流程：

- Server 為每個 key 維護 version number，每次寫入時遞增，把 version number 連同值一起儲存
- Client 讀取時，server 回傳所有未被覆蓋的值和最新 version number

buildneat.org

- Client 寫入時，必須帶上前次讀取的 version number，並合併前次讀取收到的所有值
- Server 收到帶 version number 的寫入時，可以覆蓋所有等於或低於該版本的值，但保留更高版本的值（因為那些是並發的）

多個 replica 時，需要的不是單一 version number，而是每個 replica 各自的 version number 組成的集合叫做 **version vector**（有時也叫 vector clock，但嚴格來說有所不同）。Version vector 從 replica 發送給 client，讀取後寫入時需要帶回去。

什麼時候在面試裡用這些

Replication 幾乎在所有分散式系統面試中都會出現。你的目標不只是背出這些術語，而是能根據需求選擇適合的架構。

讀取擴展：如果問題的核心是讀取量太高，直接提 read replica（follower）。「我會用 single-leader replication，讓所有寫入走 primary，讀取分散到多台 replica。」接著主動提 replication lag 的問題和解法。

高可用：任何需要「服務不能停」的系統都要談 replication。「如果 primary 掛掉，我們需要 automatic failover，把一台 replica 提升為新的 primary。」然後講 failover 的挑戰，像是split brain、資料遺失風險。

地理分散：多個 datacenter 的場景，主動提 multi-leader replication。「我會在每個 datacenter 放一台 leader，讓用戶寫到最近的 datacenter，減少延遲。然後要討論衝突解法。」

高可用 + 寫入擴展：提 leaderless replication 的 quorum 模型。「用 Cassandra 的 quorum 寫入（w=2，n=3），允許一個節點失效而不影響寫入；同樣的 quorum 讀取確保讀到最新值。但要注意 LWW 的資料遺失問題。」

常見面試情境

社群媒體（Twitter、Instagram）：讀遠多於寫，read replica 是第一個工具。Primary 處理所有寫入，幾台 replica 分散讀取。Replication lag 可以接受（用戶不需要即時看到所有人的最新動態），但自己剛發的貼文要馬上看到需要 read-after-write consistency，自己的內容從 primary 讀。

金融系統（支付、庫存）：強一致性比可用性更重要。Single-leader replication 搭配同步複製（或 semi-synchronous），確保每筆交易都在至少兩個節點上持久化。Failover 要謹慎處理，可能寧願停機也不要資料不一致。

全球 CDN / 快取系統：Multi-leader replication 最有說服力。每個地區有自己的 leader，用戶讀寫都打到最近的 datacenter。衝突可以用 LWW（cache 的資料遺失通常可接受），TTL 讓過時資料自然失效。

buildnoat.org

IoT / 指標收集（Metrics）：大量時序寫入，leaderless 模型（Cassandra）適合——高寫入吞吐量，LWW 的資料遺失在感測器資料的場景通常可接受，每次更新反正很快就被下一筆取代。

協作應用（Google Docs 類型）：需要 multi-leader 模型。每個用戶的本地狀態是一個 leader，變更即時套用到本地，非同步複製。衝突解法需要 Operational Transformation 或 CRDT，不是簡單的 LWW。

常見的 Deep Dive 問題

「你怎麼保證用戶寫入後馬上讀得到？」

這是 read-after-write consistency 的問題。在 leader-follower 架構下，寫入走 primary，讀取可能打到 follower，如果 follower 還沒同步，用戶就看不到自己剛寫的東西。

解法一：自己的資料從 primary 讀。如果能識別「這個資料只有這個用戶能改」（比如自己的 profile），讀取就永遠打 primary。

解法二：追蹤用戶的最後寫入位置。在 session 或 cookie 裡記錄用戶最後一次寫入的位置標記（例如 PostgreSQL 的 WAL LSN、MySQL 的 binlog position）。讀取時，只接受同步位置已經追上這個標記的 replica；如果所有 follower 都還沒追上，就從 primary 讀。

這裡的「位置標記」是 replication log 的邏輯位置，不是牆上的時鐘。用實際時鐘會有 clock skew 的問題，用 log position 就不會。每個 replica 持續從 primary 拉取 replication log，每處理完一筆就更新自己同步到的位置，所以系統隨時知道每台 replica 的進度。

```
Primary:    [LSN 1230] [LSN 1231] [LSN 1232] [LSN 1233] [LSN 1234]
Replica A:  [LSN 1230] [LSN 1231] [LSN 1232]  ← 同步到 1232，還沒追上
Replica B:  [LSN 1230] [LSN 1231] [LSN 1232] [LSN 1233] [LSN 1234]  ← 已追上
→ Client 帶著 LSN 1234，Replica A 不夠新，路由到 Replica B 或 Primary
```

解法三：短暫的讀 primary 視窗。用戶剛寫入後的一小段時間（比如 5 秒），所有讀取都打 primary，過了這段時間再回到 follower。簡單但有點浪費。

buildmeat.org

> **面試提示**：面試官問這個問題通常是想看你有沒有意識到 replication lag 的問題，以及你有沒有實際可行的解法。先承認問題，再提解法，然後討論各解法的取捨。

「Failover 時怎麼避免資料遺失？」

非同步 replication 下，primary 掛掉前可能有些寫入還沒複製到 follower。把一台 follower 提升為 primary 後，這些寫入就消失了。

同步 replication（最強保證）：Primary 等到至少一台 follower 確認才算寫入成功（semi-synchronous）。這樣 failover 時，那台同步 follower 保證有最新資料。代價是寫入延遲增加，而且那台同步 follower 掛了，寫入就卡住。

追蹤最新 follower：選出的新 primary 永遠是資料最新的那台。在非同步系統中這最小化資料遺失，但不是完全沒有遺失。

Raft / Paxos 等 consensus 演算法：真正的強一致性需要 consensus，以便確保多數節點在任何提交的寫入上達成共識。PostgreSQL 的 Patroni、etcd 等工具實作了這套機制。代價是複雜度和延遲增加。

「Multi-Leader 的寫入衝突要怎麼設計解法？」

這是面試官喜歡深挖的問題，因為沒有完美答案，你展現的是對取捨的理解。

先問清楚一致性需求：這個資料的衝突有多嚴重？金融交易的衝突絕對不能丟，社群媒體 profile 的衝突也許可以用 LWW。

LWW（Last Write Wins）：最簡單，但丟資料。只適合資料遺失可接受的場景（caching、用戶設定偏好等）。你需要告訴面試官你清楚這個代價。

衝突迴避：把特定記錄的所有寫入路由到同一台 leader，從根本上避免衝突。代價是把一些操作變成跨 datacenter 的寫入，增加延遲。

保留所有版本 + 應用層解決：把所有衝突版本都存下來，讓應用程式讀取時決定怎麼合併。對購物車類的資料（可以取聯集）很適合，但需要應用程式額外邏輯，而且 tombstone 的處理（標記刪除）也有複雜度。

CRDT：如果資料結構允許（計數器、集合、有序列表），用 CRDT 可以自動合併，沒有衝突。Riak 2.0 原生支援，Cassandra 也有計數器類型。

「Quorum 讀寫在網路分區時怎麼辦？」

這是 CAP 理論的核心問題。當網路分區（partition）發生時，你必須選擇：繼續提供服務（保可用性，犧牲一致性），還是拒絕服務直到分區修復（保一致性，犧牲可用性）。

CP 配置（一致性 > 可用性）：設定嚴格的 quorum，當連不到足夠的節點時就返回錯誤。你的寫入永遠不會有資料遺失，但在分區期間系統可能無法服務。適合金融系

buildmoat.org

統。

AP 配置（可用性 > 一致性）：用 sloppy quorum，即使連不到「正確的」n 個節點，也接受寫入，先存到能連到的節點上，等網路修復再 hinted handoff 回原本的節點。系統一直可以寫，但可能讀到過時資料。適合用戶行為記錄、IoT 資料等。

> **面試提示**：面試官通常希望你說出這個取捨，而不是假裝有完美的解法。「你是選 CP 還是 AP？根據你的需求，這兩個都有合理的答案。」這種回答比「用 quorum 就沒問題了」更有深度。

系統設計面試如何提到 Replication

主動帶進議題，不要等面試官問

厲害的面試者不會等到被問才提 replication。在設計讀取密集系統時，主動說：「這個讀取量很高，我會加 read replica 來分散負載，不過需要討論 replication lag 的問題。」在設計需要高可用的系統時，主動說：「單點的 primary 是 SPOF，我會設計 automatic failover，讓某台 replica 在 primary 失效時接管。」

永遠討論取捨

提到 replication 時，永遠附上取捨：

- **Read replica** → 「可以擴展讀取，但有 replication lag，要處理 read-after-write consistency」
- **Synchronous replication** → 「不會丟資料，但寫入延遲增加，那台 follower 掛了寫入就卡住」
- **Multi-leader** → 「多個 datacenter 可以各自接受寫入，但有寫入衝突需要解決」
- **Leaderless** → 「高可用寫入，但一致性模型更弱，LWW 可能丟資料」

用數字支撐決策

「這個系統每秒有 50,000 次讀取，遠超過單台 DB 能處理的上限，所以我會加三台 read replica，把讀取分散出去。Replica 之間的 replication lag 通常在 100ms 以內，對這個場景可以接受。」

面試高頻話術範例

> 「我會用 leader-follower replication，所有寫入走 primary，三台 replica 分散讀取。因為是非同步 replication，follower 可能有幾秒的 lag，對大多數讀取可以接受，但用戶剛發的貼文要馬上看到，所以我會在用戶自己的 session 裡追蹤最後寫入的 timestamp，在那段時間內強制從 primary 讀。」

buildnoat.org

> 「為了高可用，我會設計 automatic failover：每個 follower 都有 replication lag 的 metric，一旦 primary 超過 30 秒沒有心跳，lag 最小的 follower 被提升為 primary，client 端的連線池自動重導向。要注意 split brain 的風險，所以我會用 fencing token 確保舊的 primary 完全下線後才讓新的 primary 接管。」

總結

Replication 是分散式系統設計的核心問題，幾乎出現在每一個需要高可用或高吞吐量的系統面試中。

三種主要架構的核心取捨：

架構	優點	缺點	適用場景
Single-Leader	簡單、無衝突	Leader 是單點、所有寫入都走同一個節點	大多數讀取密集應用
Multi-Leader	跨 datacenter 寫入延遲低、容忍 datacenter 失效	寫入衝突需要解決、實作複雜	多 datacenter 部署、離線操作
Leaderless	高可用寫入、容忍節點失效	一致性模型弱、需要應用層合併	高可用需求、能接受最終一致性

Replication Lag 是你必須深入理解的核心問題：Read-after-write consistency、monotonic reads、consistent prefix reads，這三個一致性保證不是靠架構本身自動獲得的，需要你在設計中明確處理。

沒有「最好的」架構：面試官在乎的是你能不能根據需求做出有根據的選擇，解釋清楚你接受的取捨，以及你知道哪些保證你的設計沒有提供。「我選擇非同步 replication，接受可能最多幾秒的 lag，換取更低的寫入延遲和更高的可用性，在這個社群媒體的場景，這個取捨是合理的。」這種說法，比「我用 replication，所以系統可以 scale」要有力得多。

自我測驗

Q1：Single-Leader、Multi-Leader、Leaderless 三種架構各自最大的優點和缺點？

> - **Single-Leader**：簡單、無寫入衝突，但 Leader 是單點故障，所有寫入都走同一節點
> - **Multi-Leader**：跨資料中心寫入延遲低、DC 故障容錯好，但寫入衝突處理複雜
> - **Leaderless**：寫入可用性高、容忍節點故障，但一致性弱、需應用層處理合併邏輯

buildnest.org

Q2：什麼是 Read-After-Write Inconsistency？列舉兩個解法。

> **Read-After-Write Inconsistency**：使用者寫入資料後，立刻讀取卻讀到舊值，因為被路由到尚未同步的 follower。
>
> 解法：
>
> 1. **讀自己可能改過的東西時，從 leader 讀**（如自己看自己的 profile 永遠從 leader 讀）。
> 2. **追蹤最後寫入時間**，短時間內強制從 leader 讀取。

Q3：Leaderless 架構中，Quorum 的條件公式是什麼？n=3 時，w 和 r 通常設多少？

> 公式：**w + r > n**（寫入節點數 + 讀取節點數 > 總副本數），確保讀寫的節點有交集。
>
> n=3 時，通常 **w=2, r=2**（容忍一個節點失效）。

Q4：以下哪個不是 Single-Leader Failover 的挑戰？(A) Split brain (B) Conflict resolution (C) 非同步複製的資料遺失 (D) Timeout 門檻設定困難

> **答案：(B) Conflict Resolution**。
>
> Conflict resolution 是 Multi-Leader 架構的挑戰。Single-Leader 只有一個 leader 接受寫入，不會有寫入衝突。

Q5：同步 replication 和非同步 replication 的核心取捨是什麼？

> • **同步 replication**：保證 follower 有最新資料（一致性），但如果 follower 沒回應，整個系統的寫入會卡住（犧牲可用性）。
> • **非同步 replication**：寫入更快、可用性更高，但如果 leader 在 follower 同步前就掛了，那些寫入永久消失（有資料遺失風險）。
>
> 常見折衷：**半同步（Semi-synchronous）**——一台 follower 同步、其他非同步，確保至少兩份資料。

Q6：什麼是 Sloppy Quorum 和 Hinted Handoff？

> • **Sloppy Quorum**：當原本應該存資料的 n 台「家」節點連不到時，接受寫入到其他可達的節點上（不在原本的 n 台裡），提升寫入可用性。
> • **Hinted Handoff**：網路恢復後，這些臨時保管的寫入被送回「家」節點。
>
> 代價：即使 w + r > n，也無法保證讀到最新值，因為最新值可能在臨時保管的節點上。

buildmoai.org